MSE800 Professional Software Development

Assessment 1

SmartDrive Car Rental System

User and Technical Documentation

Student Name: Fan Fan

Student ID: 270958057

Student ID: 270958057@yoobeestudent.ac.nz

Abstract

This document explains the SmartDrive Car Rental System, developed for Assessment 1 in MSE800 Professional Software Development.

SmartDrive is a Python command-line application with an SQLite database. Customers can register, log in, browse cars, create bookings, use the Smart Booking Assistant, pay approved bookings, and review their own booking history. Admin users can manage cars, review bookings, and maintain user records.

For users, the document explains how to set up, run, and operate the program.

For assessors and developers, it explains the project structure, design diagrams, implementation choices, testing evidence, limits, and future maintenance plan.

Table of Contents

Abstract 2
Table of Contents 3
1. Introduction and Background 5
1.1 Project Background 5
2. System Overview 5
2.1 User Roles 6
2.2 Main Functional Areas 6
3. Design and Architecture (OOP Focus) 6
3.1 Programming Structure 6
3.2 Object-Oriented Structure 7
3.3 Design Diagrams 8
3.3.1 ER Diagram 8
3.3.2 Use Case Diagram 9
3.3.3 Class Diagram 9
3.3.4 Sequence Diagram 10
3.3.5 Standard Booking Activity Diagram 11
3.3.6 Smart Booking Activity Diagram 12
4. Environment and Installation Guide 14
4.1 Core Technologies 14
4.2 Project Location 14
4.3 Open the Project 14
4.6 Run the Application 14
4.6.1 Default Admin Account 14
4.7 Database Initialization 14
5. User Operating Guide 15
5.1 Guest Menu 15
Register Customer 15
Login 15
5.2 Customer Operating Guide 15
View Available Cars 15
Create Booking 16

Smart Booking Assistant 16
View My Bookings 16
Pay for Booking 16
Cancel Booking 17
5.3 Admin Operating Guide 17
Manage Cars 17
Manage Bookings 17
Manage Users 17
6. Code Implementation and File Guide 18
6.1 Implementation Notes 19
6.2 Menu and Feature Implementation Examples 19
Customer Menu in the CLI Layer 19
Booking Creation in the Service Layer 20
Smart Booking Recommendation Logic 20
Payment Rule and Approval Dependency 21
Admin Review Workflow 21
7. Testing and Results 21
7.1 Manual Testing 21
7.2 Result Summary 22
8. Known Issues and Limitations 22
9. Software Evolution Plan 22
9.1 Handling Updates and Bug Fixes 22
9.2 Adding New Features 23
9.3 Versioning Strategy 23
9.4 Backward Compatibility 24
9.5 Long-Term Software Engineering Value 24
10. Conclusion 25
10.1 Future Direction 25
11. Licensing Terms 25
12. References 25
13. Appendix 25
13.1 Representative Manual Test Output 26
13.2 Supporting Notes 26

1. Introduction and Background

SmartDrive is a console-based car rental application. It was built as a practical course project, with enough structure to model a small rental business without making the scope too large.

This document works as both a user guide and a technical guide. It explains how the system is used, how the main files fit together, and how the design supports the final implementation.

1.1 Project Background

A car rental system has to keep several records in sync. It needs user accounts, vehicle availability, booking requests, rental rules, and admin decisions. Even in a small academic version, these parts need a clear structure.

The project uses a command-line interface on purpose. This keeps the focus on object-oriented design, database persistence, input handling, and booking logic instead of screen design.

I kept the interface simple so the main effort could go into the workflow, the class structure, and the database-backed behaviour.

2. System Overview

The project has two main parts:

Task 1 covers the standard booking flow: registration, login, car browsing, booking creation, validation, admin review, and payment.

Task 2 adds the Smart Booking Assistant, which asks a few preference questions and recommends suitable cars before the booking is created.

The application starts from app.py. On startup, it opens or creates the SQLite database, checks the schema, creates the default admin account if needed, and loads the starter car data.

A user can run the system from a clean environment without creating the database by hand. The startup code prepares the tables and starter fleet automatically, which makes the project easier to demonstrate and test.

After startup, the guest menu appears. A guest can register, log in, or exit. After login, the menu changes depending on whether the user is a customer or an admin.

- **Customer:** view available cars, create booking, use Smart Booking Assistant, view own bookings, pay for approved bookings, cancel pending booking, logout
- **Admin:** manage cars, manage bookings, manage users, logout

The main business rules sit in AuthService, CarService, and BookingService. Database work is kept in repository classes, so the CLI code does not need to manage SQL directly.

A booking starts as pending. An admin can then approve or reject it. If it is approved, the customer can pay it, and the status changes to paid.

2.1 User Roles

Role	Description	Main Capabilities
Guest	User who has not logged in yet	Register customer account, log in, exit
Customer	Registered user who wants to rent a car	View cars, create booking, use Smart Booking Assistant, view personal bookings, pay approved bookings, cancel pending booking
Admin	Administrator who manages system data and booking decisions	Manage cars, review bookings, manage users, view system records

2.2 Main Functional Areas

- User registration and login
- Car inventory management
- Booking creation and customer self-service
- Rental fee calculation with optional insurance
- Administrative approval and rejection workflow
- Payment of approved bookings
- Smart Booking Assistant recommendation flow

3. Design and Architecture (OOP Focus)

This section explains the main design choices behind SmartDrive. The project uses layers and object-oriented classes to keep data, rules, and user interaction separate.

3.1 Programming Structure

The project follows this simple layered structure:

- **Entry layer:** `app.py`
- **Presentation layer:** `cli.py`
- **Business logic layer:** `services.py`
- **Persistence layer:** `repositories.py`
- **Infrastructure layer:** `database.py` and `bootstrap.py`
- **Data model layer:** `models.py`

3.2 Object-Oriented Structure

The project uses object-oriented programming as a core design approach. Domain models hold the main data, service classes handle business rules, and repository classes handle database access.

Class	Type	Main Attributes or Responsibility
`User`	Domain model	Stores identity, contact details, password hash, and role
`Car`	Domain model	Stores rental car details such as rate, mileage, category, and availability
`Booking`	Domain model	Stores booking dates, pickup location, fee, status, and insurance choice
`AuthService`	Service class	Handles registration, login, and user account updates
`CarService`	Service class	Handles car listing, inventory changes, and recommendation logic
`BookingService`	Service class	Handles booking creation, review, payment, quote calculation, and cancellation

The three main domain objects are dataclasses in models.py. This makes their attributes easy to see and keeps the object structure readable.

```
@dataclass
class User:
    user_id: Optional[int]
    username: str
    password_hash: str
    email: str
    phone: str
    role: str

@dataclass
class Car:
    car_id: Optional[int]
    make: str
    model: str
    year: int
    mileage: int
    available_now: bool
    daily_rate: float

@dataclass
class Booking:
    booking_id: Optional[int]
    user_id: int
    car_id: int
```

```
    start_date: str
    end_date: str
    status: str
```

This class-based design keeps the code easier to follow. It also means the class diagram reflects the real implementation instead of being a separate theoretical model.

3.3 Design Diagrams

The design diagrams appear before the setup and user instructions so the reader can understand the system shape first. This makes the later installation and code sections easier to follow.

Figures 1 to 6 describe the system from different angles: data, user roles, class structure, object interaction, the standard booking process, and the Smart Booking Assistant.

3.3.1 ER Diagram

This diagram explains the data that must be stored. It focuses on the database view of the project, not the menu layer or the service layer.

As shown in Figure 1, the ER diagram contains User, Car, and Booking. It shows how bookings connect customers with cars and supports the repository queries used by SQLite.

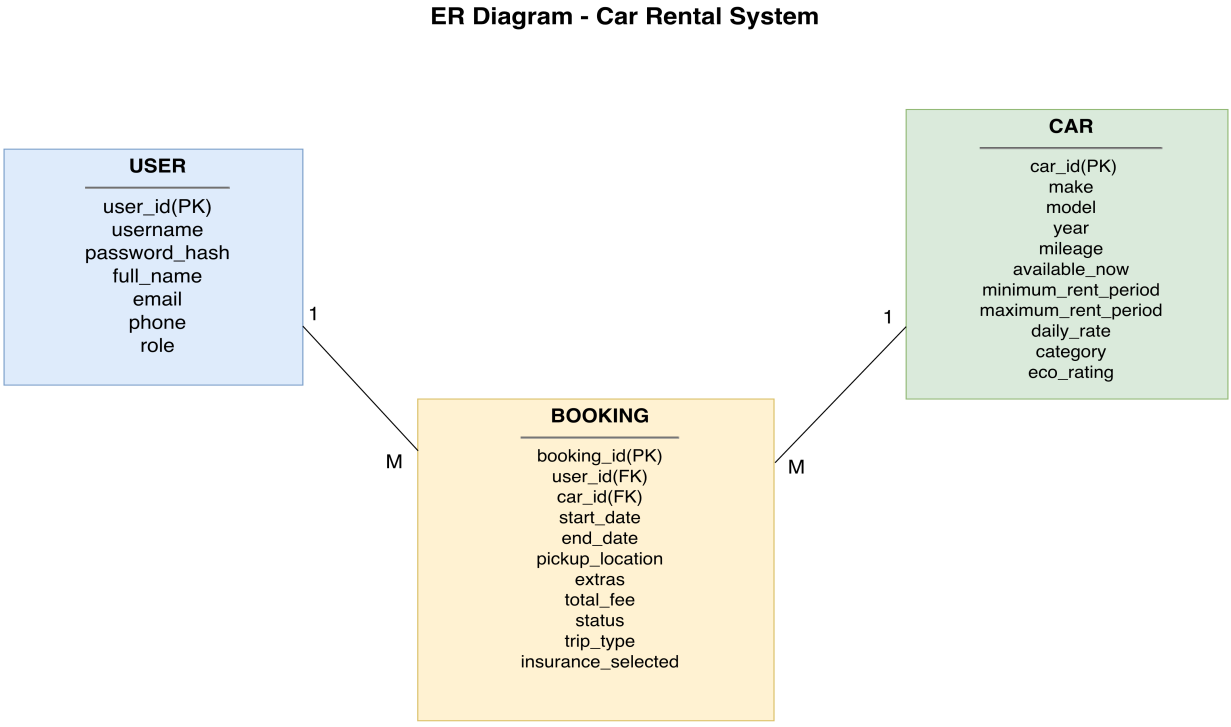


Figure 1. ER Diagram of the SmartDrive Car Rental System

3.3.2 Use Case Diagram

This diagram shows the project from the user's point of view. It is useful before reading the class design because it makes the roles and system boundaries clear.

As shown in Figure 2, Guest, Customer, and Admin have different access levels. The diagram also shows that payment belongs to the customer flow after admin approval.

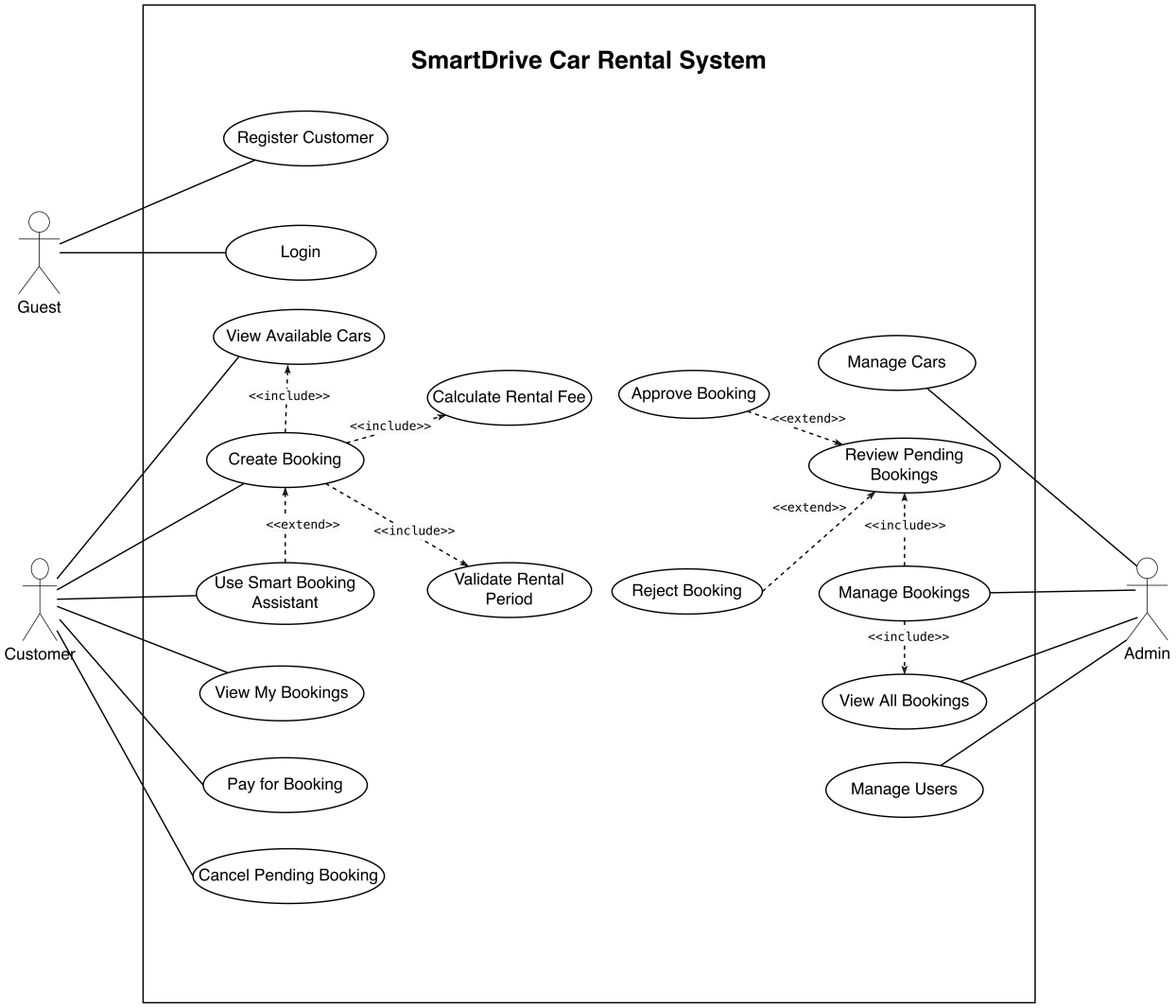


Figure 2. Use Case Diagram of User Roles and System Functions

3.3.3 Class Diagram

This diagram supports the object-oriented part of the assessment. It shows how the project is divided into domain models and service classes.

As shown in Figure 3, the main rental flow uses User, Car, Booking, and BookingService, along with supporting service classes for login, recommendations, review, and payment.

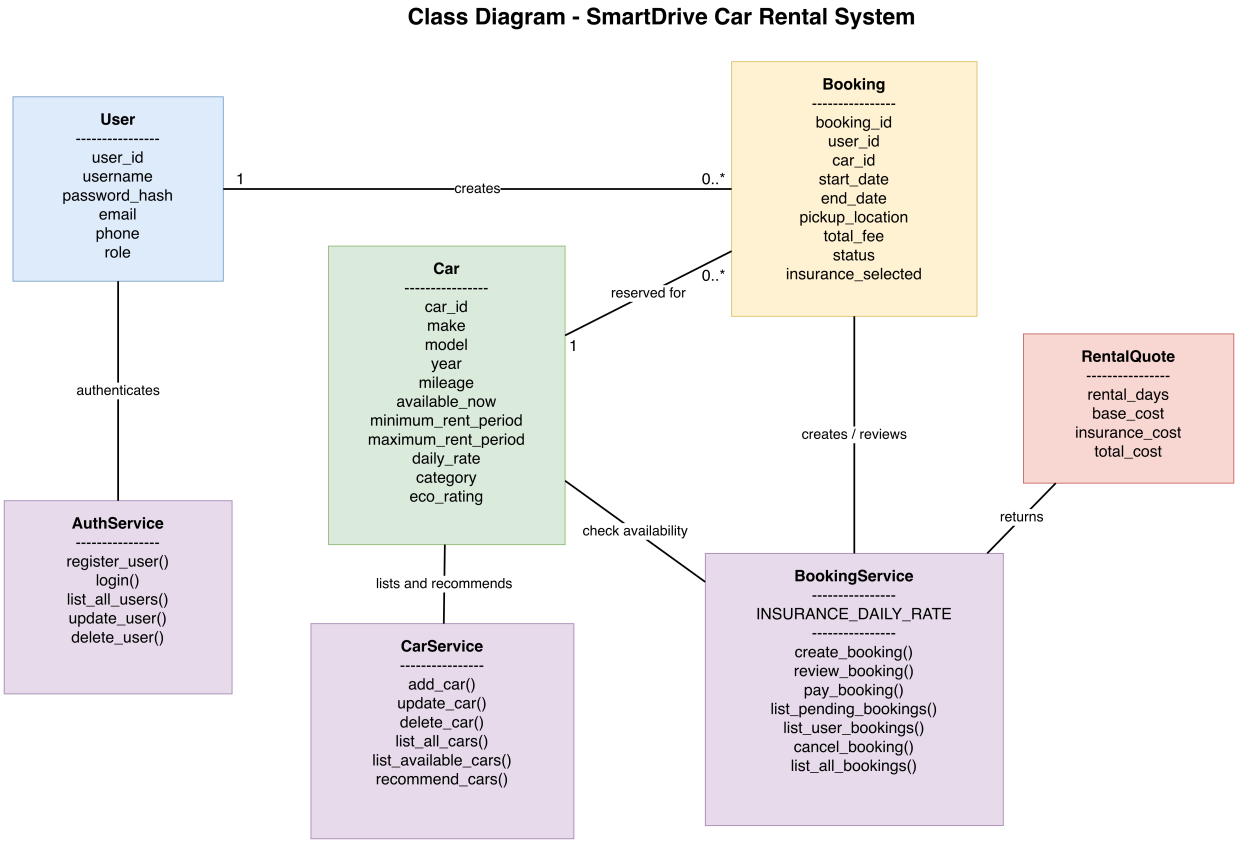


Figure 3. Class Diagram for the Main Domain and Service Classes

3.3.4 Sequence Diagram

This diagram adds the time order that the class diagram cannot show. It traces how a booking request moves from the customer to the CLI, then to the service layer and admin review.

As shown in Figure 4, the sequence diagram follows booking creation, pending status, admin approval, and customer payment. It connects the static class design with the actual runtime flow.

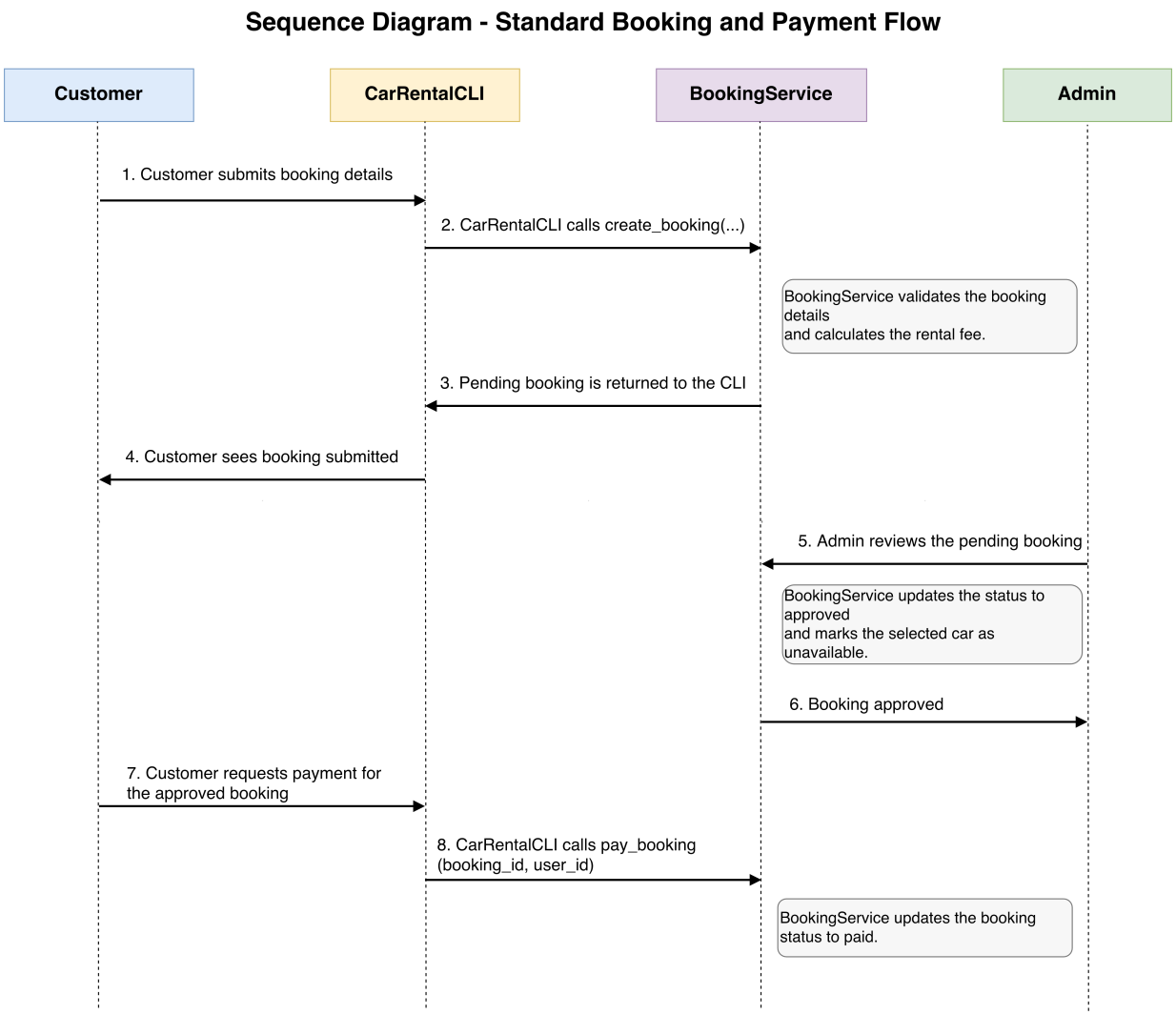


Figure 4. Sequence Diagram for Standard Booking and Payment Flow

3.3.5 Standard Booking Activity Diagram

This activity diagram shows the normal booking process as a business workflow. It focuses on actions and decisions rather than object messages.

As shown in Figure 5, the standard flow starts with booking creation, moves through validation and admin review, and ends with payment when the booking is approved.

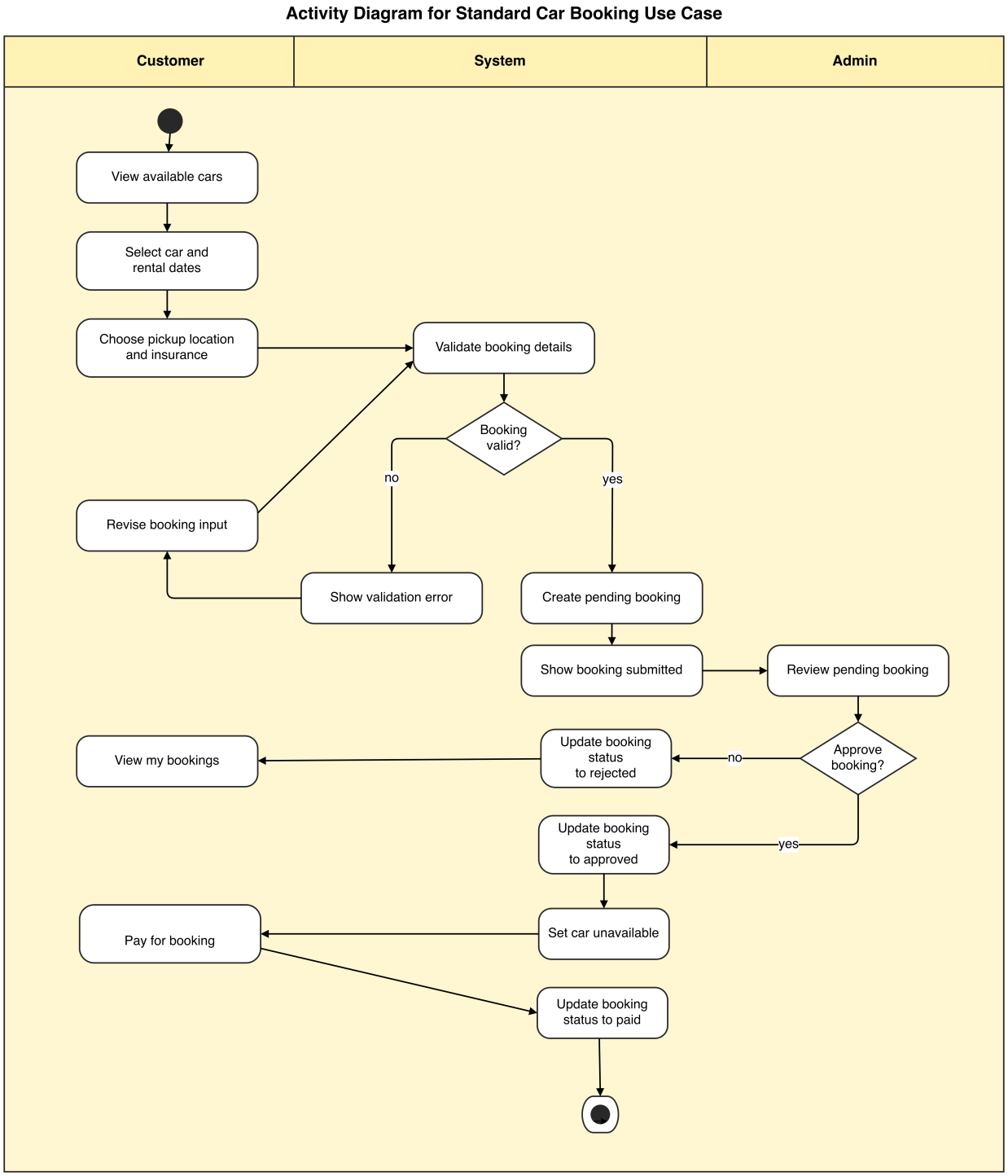


Figure 5. Standard Booking Activity Diagram

3.3.6 Smart Booking Activity Diagram

This diagram documents the innovative part of the project separately. The Smart Booking Assistant has its own flow because it changes how customers choose a car.

As shown in Figure 6, the assistant collects customer preferences, recommends suitable cars, and then leads into booking creation and payment.

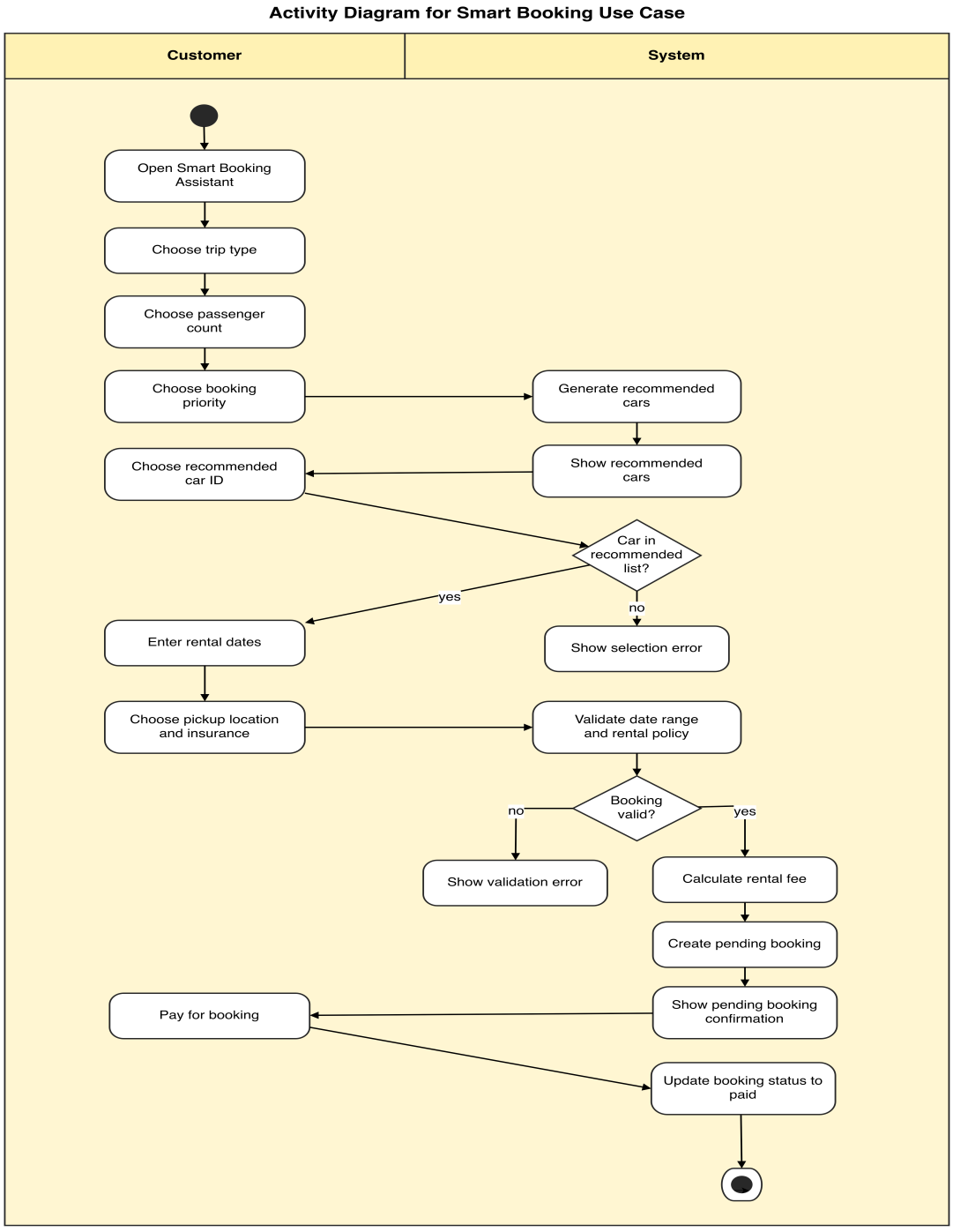


Figure 6. Smart Booking Activity Diagram

4. Environment and Installation Guide

The application is lightweight. It uses Python and SQLite, and it can run locally without extra packages for the core rental functions.

4.1 Core Technologies

Technology	Purpose in This Project
Python 3.8+	Main programming language for all application logic
SQLite	Local relational database for persistent data storage
Command-line interface	User interaction and menu-based operation
Python standard library	Hashing, date handling, path management, password input, and database access
Optional document packages	Used only when rebuilding submission documents

4.2 Project Location

GitHub repository: https://github.com/aimeefan99/MSE800-Assessment/tree/main/Assessment1

4.3 Open the Project

Open a terminal and move into the project folder:

```
cd MSE800-Assessment/Assessment1
```

4.6 Run the Application

Use the following command:

```
python3 app.py
```

4.6.1 Default Admin Account

The default admin account is created automatically if it is missing.

- Username: `admin`
- Password: `admin123`

4.7 Database Initialization

When the program starts, it prepares the database automatically. This includes:

- creating tables if they do not already exist

- adding missing user contact columns when opening an older database
- creating the default admin account if necessary
- seeding the default fleet of ten cars

5. User Operating Guide

5.1 Guest Menu

The guest menu appears first when the program starts.

1. Register customer
2. Login
3. Exit

Register Customer

The user provides:

- username
- email
- phone
- password

After registration, the new account is saved in the database and the user can log in as a customer.

Login

Registered users log in with a username and password. The system checks the user's role and opens either the customer menu or the admin menu.

5.2 Customer Operating Guide

The customer menu contains the following options:

4. View available cars
5. Create booking
6. Smart booking
7. View my bookings
8. Pay for booking
9. Cancel booking
10. Logout

View Available Cars

This option lists cars that are currently available. Each row shows the car ID, year, make, model, mileage, daily rate, rental period range, and availability.

Create Booking

The customer selects a car, enters dates, chooses a pickup location, and decides whether to add insurance. The system calculates the price and saves the booking as pending.

Booking validation checks:

- date order validation
- car availability check
- minimum rent period check
- maximum rent period check

Smart Booking Assistant

This feature asks the customer:

- What type of trip this is
- How many people are travelling
- What matters most, such as price, newer car, lower mileage, or rental flexibility

The system uses these answers to recommend three cars. The customer must choose one of the recommended car IDs before continuing.

This reduces guesswork. Instead of scanning the whole fleet, the customer gets a short list that better matches the trip.

The recommendation logic is implemented in CarService.recommend_cars(). Code and terminal examples are shown in the implementation section.

View My Bookings

This option shows only the current customer's bookings. Each record includes:

- booking ID
- car name
- date range
- pickup location
- total fee
- status

Pay for Booking

This option is available after an admin approves a booking. The system lists approved bookings waiting for payment. If none exist, it tells the customer there is nothing to pay yet. After payment, the booking status changes from approved to paid.

The current payment step is a simplified project feature, not a real payment gateway. It completes the rental lifecycle without adding third-party payment integration.

The payment code and terminal examples are shown in the implementation section.

Cancel Booking

A customer can cancel only their own pending bookings. Rejected, approved, and paid bookings cannot be cancelled.

5.3 Admin Operating Guide

The admin menu contains the following options:

11. Manage cars
12. Manage bookings
13. Manage users
14. Logout

Manage Cars

This submenu includes:

15. View all cars
16. Add a car
17. Update a car
18. Delete a car
19. Back

The admin can maintain the fleet from this menu. When an invalid car ID is entered, the system prompts the admin to try again where appropriate.

Manage Bookings

This submenu includes:

20. View all bookings
21. Review pending bookings
22. Back

This menu lets the admin approve or reject pending bookings. When a booking is approved, the selected car becomes unavailable to later customer searches.

This manual review step reflects a small rental business. It lets the admin confirm a request before the customer can pay.

Manage Users

This submenu includes:

23. View all users
24. Add a user
25. Update a user
26. Delete a user
27. Back

The admin can add, update, and remove user records. The system protects the default admin account and prevents the current admin from deleting their own account.

6. Code Implementation and File Guide

The simplified tree below shows the main project layout.

```
Assessment1/
├── README.md
├── requirements.txt
├── app.py
├── cli.py
├── models.py
├── services.py
├── repositories.py
├── database.py
├── bootstrap.py
├── decorators.py
├── data/
│   └── car_rental.db
├── diagrams/
│   ├── *.drawio
│   └── *.png
├── docs/
│   ├── Assessment1_SmartDrive_User_and_Technical_Documentation.pdf
│   ├── Assessment1_SmartDrive_Design_and_Architecture.pdf
│   ├── Assessment1_Maintenance_and_Support.pdf
│   └── manual_test_cases.md
```

File	Purpose
`app.py`	Main program entry point
`cli.py`	Menu-driven command-line interface and user interaction flow
`services.py`	Business logic for authentication, booking, pricing, and recommendations
`repositories.py`	Database persistence operations for users, cars, and bookings
`database.py`	SQLite connection wrapper and query execution support
`bootstrap.py`	Schema creation, admin seeding, default car seeding, and compatibility upgrade logic
`models.py`	Data models for User, Car, and Booking

`decorators.py`	Contains the admin-only decorator used in admin actions

The files follow a layered structure. app.py starts the program, cli.py handles menu interaction, services.py applies business rules, repositories.py handles database work, and models.py defines the main objects.

6.1 Implementation Notes

The project includes comments where the logic benefits from explanation. The comments focus on reasoning, not on repeating obvious code.

For example:

- `bootstrap.py` explains why user contact columns are patched into older databases
- `services.py` explains the stages of smart booking recommendation scoring
- `cli.py` explains why date order is checked before later booking steps

There are also file-level and function-level comments to help readers understand each script's role.

The main features map directly to specific files. AuthService handles registration and login. CarService handles browsing and smart recommendations. BookingService handles booking, review, cancellation, and payment.

The CLI does not contain the business rules. It collects input, prints menus, and sends requests to the service layer.

This separation is clearest in booking and payment. The CLI asks for details, while BookingService decides whether the action is valid.

repositories.py keeps SQL separate from service logic. This is useful for user lookup, car availability checks, and booking status updates.

6.2 Menu and Feature Implementation Examples

This section explains why each code sample is included. The goal is not simply to show that code exists, but to connect key implementation choices with visible system behaviour. Each example is paired with terminal output so the reader can see how the code appears in the running program.

Customer Menu in the CLI Layer

This example shows how the customer-facing menu is defined in the CLI layer. It was selected because it gives a quick overview of the customer workflow and shows that payment is now part of the normal menu structure.

The main point to notice is that cli.py controls the interaction flow, while the business rules remain outside the menu code. The menu therefore acts as an entry point to the later service-layer actions rather than containing the booking logic itself.

```
def _customer_menu(self) -> None:
    print("\nCustomer Menu")
    print("1. View available cars")
    print("2. Create booking")
    print("3. Smart booking")
    print("4. View my bookings")
    print("5. Pay for booking")
    print("6. Cancel booking")
    print("7. Logout")
```

Booking Creation in the Service Layer

This code sample shows where a booking is actually created in the service layer. It was included to demonstrate that booking creation is more than a simple insert: it also depends on validation, pricing, and status assignment.

The reader should pay attention to the sequence of checks, the calculation of the final fee, and the creation of a pending booking. The terminal output below confirms that the same logic produces the expected booking summary in the live system.

```
def create_booking(self, user_id, car_id, start_date, end_date,
                   pickup_location, insurance_selected):
    car = self.car_repository.get_by_id(car_id)
    if car is None or not car.available_now:
        raise ValueError("Selected car is not available.")

    rental_days = self._calculate_days(start_date, end_date)
    quote = self._calculate_quote(car, rental_days, insurance_selected)
    booking = Booking(..., total_fee=quote.total_cost, status="pending")
    return self.booking_repository.create(booking), quote
```

Smart Booking Recommendation Logic

This example was chosen to show the core of the Smart Booking Assistant. It is one of the clearest pieces of evidence that the project goes beyond a standard rental script and includes the innovation required by Task 2.

The important part here is not only that the system returns recommendations, but that it does so through a readable rule-based method linked to trip purpose, passenger count, and customer priority. The matching terminal output shows how those rules appear to the user.

```
def recommend_cars(self, trip_purpose, passenger_count, priority):
    cars = self.car_repository.list_available()
    scored_cars = []
    for car in cars:
        score = 0
        # score by trip purpose, group size, and top priority
        ...
```

```
            scored_cars.append((score, car))
        scored_cars.sort(..., reverse=True)
        return [car for _, car in scored_cars[:3]]
```

Payment Rule and Approval Dependency

This sample explains the payment rule more clearly than a screenshot alone could. It shows that payment is not available for every booking, and that approval is a required step before the booking can move to paid status.

That rule matters because it connects the customer flow with the admin review flow. The output example underneath demonstrates both sides of the behaviour: blocked payment before approval and successful payment after approval.

```
def pay_booking(self, booking_id: int, user_id: int) -> Booking:
    booking = self.booking_repository.get_by_id(booking_id)
    if booking is None:
        raise ValueError("Booking not found.")
    if booking.user_id != user_id:
        raise ValueError("You can only pay for your own booking.")
    if booking.status != "approved":
        raise ValueError("Only approved bookings can be paid.")

    self.booking_repository.update_status(booking_id, "paid")
    return self.booking_repository.get_by_id(booking_id)
```

Admin Review Workflow

This final example was added to show the admin side of the workflow. The customer journey is only complete because the admin can review pending bookings and decide whether they move forward.

The snippet and output together show that approval is a real implemented step, not just a diagrammed idea. They also make it clear why the payment feature depends on admin review first.

7. Testing and Results

7.1 Manual Testing

Manual testing used realistic customer and admin flows. The scenarios covered:

- registering a customer account
- logging in as customer and admin
- viewing available cars
- creating a standard booking
- creating a smart booking from recommended cars
- viewing and cancelling pending bookings

- adding, updating, and deleting cars
- adding, updating, and deleting users
- approving and rejecting pending bookings
- attempting payment before approval
- paying an approved booking
- checking the final paid booking result

The full terminal record is in **docs/manual_test_cases.md**. It includes the updated payment flow, including blocked payment before approval and successful payment after approval.

7.2 Result Summary

The implemented system supports the required assessment workflow. It includes persistent storage, role separation, car inventory management, booking creation, admin review, payment, and the Smart Booking Assistant.

Manual testing matters here because the application is menu-driven. Many behaviours depend on the order of user actions, so the terminal records help show that the flow works end to end.

8. Known Issues and Limitations

- The application is CLI-based and does not provide a GUI or web interface.
- Smart Booking Assistant is rule-based and intentionally lightweight.
- Phone and email validation are currently limited.
- One car record represents one physical vehicle rather than a fleet stock count.
- Payment is not integrated with a real payment gateway.
- The current payment function is a simplified workflow step and does not include receipt generation, payment method selection, or transaction history.

These limits fit the assessment scope. The project demonstrates structure, persistence, and booking flow rather than every feature of a commercial rental platform.

9. Software Evolution Plan

This section addresses **Task 3**: Software Evolution (LO2). It explains how SmartDrive can be updated, maintained, versioned, and kept compatible with older data.

The project is small, but it can grow in stages. New features can usually be added in the right layer instead of rewriting the whole codebase.

9.1 Handling Updates and Bug Fixes

Area	Recommended Practice	Suitable Tool or Method
Bug recording	Log each defect with affected workflow, expected	GitHub Issues

	behaviour, and verification notes	
Code change review	Make changes in a separate branch and review them before merging	GitHub branches and pull requests
Documentation updates	Update diagrams, README, and user document when behaviour changes	Repository documentation plus shared notes
Team-scale project tracking	Track assigned work, priorities, and progress across iterations	Jira, with Confluence for supporting documentation

Small updates should start by finding the affected layer. A pricing issue belongs in services.py. A data retrieval issue belongs in repositories.py. A menu text issue belongs in cli.py.

Each change should begin with a recorded issue. For this project, GitHub Issues would be enough to track bugs, feature requests, and maintenance tasks.

For a small project, GitHub Issues and pull requests are usually enough. In a larger team, the same process could be managed with Jira for task tracking and Confluence for decisions and support notes.

9.2 Adding New Features

Future features can be added in stages. Payment history, overlap checks, and richer recommendation rules could all be added without replacing the current structure.

A practical workflow is to create a feature branch, make the change in the correct layer, test the affected flow, and then merge it back through a reviewed pull request.

This lowers risk because each change is small enough to review. It also makes it easier to see how a change affects booking creation, recommendations, admin review, or payment status.

9.3 Versioning Strategy

Version Type	Meaning in This Project	Example
Patch	Small fixes that do not change the user workflow	`v1.0.1`
Minor	Visible new features or controlled behaviour improvements	`v1.1.0`
Major	Large redesign of structure, data flow, or interaction model	`v2.0.0`

A simple versioning approach is enough for this project. Bug fixes can be patch releases, while visible features such as payment or smart booking can be minor releases.

A lightweight semantic versioning style would work well:

- **Patch version** changes for small fixes that do not change the user workflow
- **Minor version** changes for new features such as payment support or smarter recommendations
- **Major version** changes for significant redesigns to structure, data flow, or user interaction

Each version can be marked with a GitHub tag and a short release note. A changelog should record what changed, what was fixed, and whether the database was affected.

9.4 Backward Compatibility

Compatibility Concern	Current or Recommended Response
Older SQLite files missing newer columns	Patch the schema during startup in `bootstrap.py`
New booking status values	Introduce them carefully and document the behaviour change in release notes
Future database changes	Prefer additive changes before destructive schema redesign

Backward compatibility already matters because older SQLite databases may not match the latest schema. The bootstrap logic helps by checking the database and adding missing user contact columns at startup.

Future updates should introduce new fields and status values carefully. Migration logic should stay in the startup process so older data files can still be opened.

Schema changes should be additive where possible. Adding a column is safer than renaming or deleting one immediately. If a behaviour changes, the README and manual tests should be updated at the same time.

9.5 Long-Term Software Engineering Value

The system will be easier to maintain if responsibilities stay separated and names stay consistent. Clear workflows and documentation also make future changes less risky.

In practice, maintainability is not only about code. It also depends on version control, issue tracking, documented decisions, and testing after each meaningful change.

10. Conclusion

SmartDrive meets the main assessment goals by combining user management, car management, booking, admin approval, and payment in one structured Python application.

The final result is more than a running script. It is a small layered system with persistent data, role-based menus, documented workflows, and a separate smart booking path.

10.1 Future Direction

- Improve retry behavior in more customer-side inputs
- Strengthen email and phone format validation
- Add richer conflict checking for overlapping bookings
- Expand Smart Booking Assistant with more refined recommendation rules
- Extend the current mock payment step into a richer payment module with method selection, receipts, and payment history
- Create a graphical or web-based version of the system
- Expand the documented verification scenarios

These improvements would make the system more complete. The current version already gives a clear working demonstration of the assessment requirements.

11. Licensing Terms

This project is provided for educational use as part of coursework and assessment submission. It is not distributed as a commercial software product.

12. References

- Python Software Foundation. *Python Documentation*. Available at: `https://docs.python.org/3/`
- SQLite Consortium. *SQLite Documentation*. Available at: `https://www.sqlite.org/docs.html`
- Larman, Craig. *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and the Unified Process*, Second Edition. Available at: `https://learning.oreilly.com/library/view/applying-uml-and/0130925691/`

13. Appendix

This appendix lists supporting project records that are useful for checking the final submission.

Supporting File	Purpose
`docs/manual_test_cases.md`	Full manual CLI test record covering registration, booking, admin review, and payment

`diagrams/*.drawio`	Editable source files for the submitted diagrams
`diagrams/*.png`	Exported diagram images used in the README and this document

13.1 Representative Manual Test Output

The full test report is stored in docs/manual_test_cases.md. The short extract below shows the final approval-to-payment flow.

```
Customer Menu
1. View available cars
2. Create booking
3. Smart booking
4. View my bookings
5. Pay for booking
6. Cancel booking
7. Logout
Choose an option: 5
[4] 2022 Mitsubishi Outlander | 2026-05-01 to 2026-05-14 | Auckland CBD |
total=$1708.00 | status=approved
Booking ID to pay: 4
Booking #4 paid successfully.
```

13.2 Supporting Notes

The appendix is intentionally brief. Larger raw records, such as the full manual test log, are kept in separate project files so the main document stays readable.